SPH Implementation Audit: `Assets/Scripts/Simulation`

Contents

1 Scope and Method **1**

2 Spatial Discretization and Neighbor Search **2**

3 Smoothing Kernels **3**

4 Density and Near-Density Fields **3**

5 Pressure Closure and Near-Pressure Stabilization **4**

6 Viscosity **5**

7 Time Integration **5**

8 Boundary Handling **6**

9 GPU-Specific Structure **6**

10 Extensions Beyond Classical SPH: Foam, Spray, and Bubbles **6**

11 Volumetric Density Resampling **7**

12 Summary Table **7**

13 Not Implemented / Possible Improvements **8**

1 Scope and Method

This document audits the 3D fluid solver in `Assets/Scripts/Simulation` against the theory collected in `sph_study_notes.tex` (referred to below simply as the *study notes*). The reviewed source files are:

- `FluidSim.cs` — MonoBehaviour driving the simulation loop, buffer management, and per-frame parameter upload.
- `Spawner3D.cs` — initial particle placement.
- `Compute/FluidSim.compute` — all GPU kernels (external forces, spatial hashing, reordering, density, pressure, viscosity, position integration, density-texture rendering, and foam/spray/bubble particles).

- `Compute/FluidMaths3D.hlsl` — smoothing-kernel definitions.
- `Compute/SpatialHash3D.hlsl` — uniform-grid cell hashing helpers.
- `Assets/Scripts/Helpers/SpatialHash/SpatialHash.cs` — (outside `Simulation`, but instantiated directly by `FluidSim.cs`) wraps the GPU counting sort used to build the neighbor lookup table.

For each SPH concept, the exact implementation is identified with file and line references, and cross-referenced to the corresponding section of the study notes. The final section lists mechanisms described in the study notes (or standard in the wider SPH literature) that are *not* present in this codebase, together with suggested improvements.

2 Spatial Discretization and Neighbor Search

The study notes describe discretization abstractly (Section 4, particularly *What Is Discretization?*, study notes lines 540–550) but do not detail a specific neighbor-search algorithm; they point to Ihmsen et al. (2014), *SPH Fluids in Computer Graphics*, as the reference survey for neighborhood search (study notes lines 994–999).

The implementation uses a **uniform background grid with spatial hashing**, the standard approach surveyed in that paper:

- **Cell indexing.** `SpatialHash3D.hlsl`, line(s) `46--48` (`GetCell3D`) maps a world position to an integer cell coordinate by dividing by the smoothing radius, i.e. cell size equals the kernel support h, matching the usual convention that at most the 27 cells in a $3 \times 3 \times 3$ block around a particle can contain neighbors within h.
- **Hashing.** `SpatialHash3D.hlsl`, line(s) `53--63` (`HashCell3D`) hashes the cell coordinate using large prime multipliers (lines 41–43) combined with a block-wise wrap-around (block size 50) to reduce large-coordinate overflow issues, then `SpatialHash3D.hlsl`, line(s) `65--68` (`KeyFromHash`) reduces the hash modulo the particle count to produce a bounded table key.
- **27-cell neighbor stencil.** `SpatialHash3D.hlsl`, line(s) `1--39` defines the fixed `offsets3D[27]` array enumerating every neighboring cell offset in 3D.
- **Key generation kernel.** `FluidSim.compute`, line(s) `146--164` (`UpdateSpatialHash`) computes a key per particle from its *predicted* position (see Section 7).
- **Sort and offset table.** The keys are sorted with a GPU counting sort and converted into a start-offset table by `SpatialHash.Run` (`Helpers/SpatialHash/SpatialHash.cs`, lines 30–36), which calls `GPUCountSort` and `SpatialOffsetCalculator`. This produces, for each hashed key, the index at which that key's particle run begins in the sorted array — the same "counting sort into an offset table" neighbor-search strategy discussed by Ihmsen et al.
- **Cache-coherent reordering.** `FluidSim.compute`, line(s) `165--185` (`Reorder` and `ReorderCopyBack`) physically rearrange `Positions`, `PredictedPositions`, and `Velocities` into sorted order so that later neighbor loops read contiguous memory instead of indirecting through `SortedIndices` on every access. This is a GPU-performance optimization with no direct analogue in the study notes, but it materially affects achievable particle counts.

- **Neighbor traversal.** The shared pattern used in `CalculateDensities`, `CalculatePressureForce`, `CalculateViscosity`, `UpdateDensityTexture`, and `UpdateWhiteParticles` (e.g. `FluidSim.compute`, line(s) `187--226`) loops over the 27 neighboring cells, looks up each cell's start offset, and walks forward through the sorted array until the key changes, discarding neighbors outside the squared smoothing radius.

3 Smoothing Kernels

The study notes derive and verify a single kernel in detail: the **cubic B-spline (M_4) kernel** (Section 4.2.3, study notes lines 676–753), including its normalization constants $\sigma_1, \sigma_2, \sigma_3$ and its piecewise gradient (study notes lines 851–863). The implementation does *not* use the cubic spline. Instead, `FluidMaths3D.hlsl` implements the kernel family popularized by Müller, Charypar, and Gross (2003) (cited in study notes lines 950–955):

- **Poly6**, `FluidMaths3D.hlsl`, line(s) `17--26` (`SmoothingKernelPoly6`): $W \propto (h^2 - r^2)^3$. Used only by the viscosity kernel (`FluidSim.compute`, line(s) `343, 490--509`); see Section 6.

- **Spiky, power 2**, `FluidMaths3D.hlsl`, line(s) `39--47` (`SpikyKernelPow2`) and its derivative `FluidMaths3D.hlsl`, line(s) `59--67` (`DerivativeSpikyPow2`): used as the *density* kernel (`FluidMaths3D.hlsl`, line(s) `69--73`, `DensityKernel`/`DensityDerivative`). Using a Spiky-type kernel (non-zero derivative as $r \to 0$) for density, rather than the smoother Poly6 kernel that is more conventional for density summation, is a deliberate anti-clustering choice: Poly6's derivative vanishes at $r = 0$, which lets particles approach arbitrarily closely without a growing density gradient to push them apart, whereas Spiky's non-vanishing derivative keeps the density gradient finite and repulsive at small separations.

- **Spiky, power 3**, `FluidMaths3D.hlsl`, line(s) `28--36` (`SpikyKernelPow3`) and its derivative `FluidMaths3D.hlsl`, line(s) `49--57` (`DerivativeSpikyPow3`): used as the *near-density* kernel (`FluidMaths3D.hlsl`, line(s) `75--83`, `NearDensityKernel`/`NearDensityDerivative`); see Section 5.

- **Linear kernel**, `FluidMaths3D.hlsl`, line(s) `8--15` (`LinearKernel`): defined but *never called* anywhere in `FluidSim.compute`. Dead code / unused leftover.

All normalization constants (`K_SpikyPow2`, `K_SpikyPow3`, `K_SpikyPow2Grad`, `K_SpikyPow3Grad`) are declared uninitialized in the shader (`FluidMaths3D.hlsl`, line(s) `3--6`) and are uploaded once per frame from the CPU side by `UpdateSmoothingConstants` (`FluidSim.cs`, line(s) `339--351`), which recomputes them only when `smoothingRadius` changes. This mirrors the study notes' general normalization procedure (study notes lines 716–730) but specialized in closed form to the Spiky kernels rather than computed via the generic integral $\int_0^1 f(q) q^{d-1} dq$.

Notably, the implementation never explicitly includes a particle mass m_j in any kernel sum (see Section 13); all sums are of the unweighted kernel value, which is only correct if every particle is assumed to carry equal, implicitly-unit mass.

4 Density and Near-Density Fields

`FluidSim.compute`, line(s) `187--226` (`CalculateDensitiesAtPoint`) and the wrapping kernel `FluidSim.compute`, line(s) `229--241` (`CalculateDensities`) implement the **summation density**

approach described in the study notes,

$$\rho_i \approx \sum_j m_j W_{ij}$$

(study notes lines 822–827), but with $m_j \equiv 1$ and $W_{ij} =$ `DensityKernel`. Alongside the ordinary density, a second **near-density** field $\rho_i^{\text{near}} = \sum_j$ `NearDensityKernel`(r_{ij}, h) is accumulated in the same loop and returned as the second component of the `float2` result (`FluidSim.compute`, line(s) `222--224`). Both fields are evaluated using *predicted* positions, not the current positions (see Section 7).

This dual density/near-density scheme is not discussed anywhere in the study notes or its reading list; it is a lineage from Clavet, Lowe, and Beaudoin's *Particle-based Viscoelastic Fluid Simulation* (2005) "double density relaxation" technique, adapted here into a conventional force-based (rather than position-relaxation-based) SPH solver. The study notes' paper list (Section 5) jumps from Müller et al. 2003 directly to Monaghan 2005 and Becker–Teschner 2007 (study notes lines 950–969) without covering this intermediate near-pressure technique, so this is a method present in the code but absent from the notes.

5 Pressure Closure and Near-Pressure Stabilization

The study notes' Section 2.1.4 (*Pressure Closure*, study notes lines 260–292) presents two closures: WCSPH with Tait's equation $p = \frac{\rho_0 c_0^2}{\gamma}\left[(\rho/\rho_0)^\gamma - 1\right]$, and incompressible/projection SPH with a pressure Poisson equation. The implementation uses neither in full. Instead, `FluidSim.compute`, line(s) `85--88` (`PressureFromDensity`) implements the *linearized* equation of state

$$p_i = (\rho_i - \rho_{\text{target}}) \cdot k_{\text{pressure}},$$

i.e. Tait's equation with $\gamma = 1$ and no reference speed of sound, exposed as `targetDensity` and `pressureMultiplier` (`FluidSim.cs`, line(s) `22--23`). This is the simplified stiffness relation used in Müller et al. (2003) rather than the more general exponentiated Tait form the study notes write out — the notes describe the general WCSPH family (study notes lines 437–443 also discuss the γ exponent) but the code implements only its simplest linear member.

A second, independent pressure-like term, `NearPressureFromDensity` (`FluidSim.compute`, line(s) `90--93`), computes $p_i^{\text{near}} = \rho_i^{\text{near}} \cdot k_{\text{near}}$ (`nearPressureMultiplier`, `FluidSim.cs`, line(s) `24`) from the near-density field of Section 4. Both pressures are combined, symmetrized between particle pairs, and converted into a force in `FluidSim.compute`, line(s) `267--346` (`CalculatePressureForce`):

$$\mathbf{f}_i^{\text{pressure}} = -\sum_j \nabla_i W_{ij} \frac{p_i + p_j}{2\rho_j} - \sum_j \nabla_i W_{ij}^{\text{near}} \frac{p_i^{\text{near}} + p_j^{\text{near}}}{2\rho_j^{\text{near}}}$$

(signs folded into the kernel-derivative helper functions; `FluidSim.compute`, line(s) `329--331`). Using the *averaged* pressure $(p_i + p_j)/2$ for the pair force is exactly the symmetric pairwise formulation the study notes credit with exact momentum conservation (study notes lines 104–107, *Conservation and coupling*). The near-pressure term is an additional short-range repulsive correction that prevents the particle clustering/tensile-instability failure mode the study notes warn about under "Stability artifacts" (study notes lines 126–131), without needing the artificial-pressure or δ-SPH corrections the notes list as alternatives (study notes lines 459–463).

The resulting acceleration is applied directly to velocity (`FluidSim.compute`, line(s) `347--349`), and an ad hoc aerodynamic drag term is applied to particles with fewer than 8 neighbors (`FluidSim.compute`, line(s) `351--355`) to damp free-flying/airborne particles — an engineering stabilization with no counterpart in the study notes.

6 Viscosity

The study notes' momentum equation includes a physical viscous term $\mu\nabla^2\mathbf{v}$ (study notes lines 224–232), and Section 3.4 separately lists "XSPH velocity smoothing" as a distinct *regularization* technique, not a physical viscosity model (study notes lines 464–468). The implementation's `CalculateViscosity` kernel (`FluidSim.compute`, line(s) `409--459`) computes

$$\Delta\mathbf{v}_i = \texttt{viscosityStrength} \cdot \Delta t \sum_j (\mathbf{v}_j - \mathbf{v}_i)\, W^{\text{poly6}}(r_{ij}, h)$$

and adds it directly to particle velocity (`FluidSim.compute`, line(s) `456--458`). This is XSPH velocity averaging, not a discretized Laplacian of velocity: it pulls each particle's velocity toward the kernel-weighted average of its neighbors' velocities, rather than modeling a stress-divergence force scaled by dynamic viscosity μ. It therefore matches the regularization technique named in study notes Section 3.4 rather than the physical-viscosity discretizations referenced via Morris, Fox, and Zhu (1997) in the study notes' paper list (study notes lines 940–944). The kernel is skipped entirely when `viscosityStrength == 0` (`FluidSim.cs`, line(s) `332`), which is also the default value (`FluidSim.cs`, line(s) `25`).

7 Time Integration

The study notes' Section 3.5 lists explicit Euler, leapfrog, Verlet, predictor–corrector, and Runge–Kutta as integrator choices with a CFL-type timestep restriction (study notes lines 469–494). The implementation uses a **semi-implicit (symplectic) Euler** scheme with position prediction and fixed sub-stepping:

- **Gravity and prediction**, `FluidSim.compute`, line(s) `133--144` (`ExternalForces`): velocity is updated by gravity ($\mathbf{v} \mathrel{+}= \mathbf{g}\,\Delta t$), and a *predicted* position is computed as $\mathbf{x}+\mathbf{v}\cdot\frac{1}{120}$ (a fixed 1/120 s look-ahead, independent of the actual timestep) rather than the true Δt. All density/pressure computations in this step then use these predicted positions, not the actual particle positions, so the pressure solve reacts to where particles are about to be rather than where they currently sit. This look-ahead/prediction trick is not discussed in the study notes.

- **Position integration**, `FluidSim.compute`, line(s) `461--475` (`UpdatePositions`): the real `Positions` buffer is advanced by the final velocity, $\mathbf{x} \mathrel{+}= \mathbf{v}\,\Delta t$, after which boundary collisions are resolved (Section 8).

- **Fixed sub-stepping instead of a CFL formula.** Rather than computing Δt from the stability formula in study notes lines 476–486 (which involves c_0, v_{sig}, $a_{\max}$, and ν), `RunSimulationFrame` (`FluidSim.cs`, line(s) `284--308`) simply divides the frame's delta time by a fixed, user-set `iterationsPerFrame` (`FluidSim.cs`, line(s) `18`) and runs that many identical sub-steps (`FluidSim.cs`, line(s) `290--293`). An optional frame-rate floor, `maxTimestepFPS` (`FluidSim.cs`, line(s) `17`), caps the per-frame Δt so that a slow frame runs the simulation in slow motion instead of taking a single large, unstable step — a crude but effective proxy for the notes' CFL safety coefficient C_{CFL} (study notes lines 487–489), implemented without ever measuring particle velocity or acceleration.

8 Boundary Handling

The study notes discuss boundary treatment at several points: as a limitation of SPH in general ("Solid walls are not automatically represented accurately," study notes lines 132–134), as a required initial/ boundary condition (Section 2.1.6, study notes lines 315–331), as a tunable parameter category (Section 3.6, study notes lines 496–504), and via the Adami–Hu–Adams (2012) generalized wall boundary condition in the paper list (study notes lines 988–992). All of these describe *boundary particles* (ghost, dummy, or dynamic particles that participate in the SPH sums).

The implementation does not use boundary particles at all. Collisions are resolved purely geometrically in `ResolveCollisions` (`FluidSim.compute`, line(s) `101--131`): the box is transformed into local (unit-cube) space via `worldToLocal`, any axis whose local coordinate exceeds ± 0.5 is clamped back to the boundary, and the corresponding velocity component is negated and scaled by `collisionDamping` (`FluidSim.cs`, line(s) `26`), then both are transformed back to world space. This supports an arbitrarily translated/rotated/scaled rectangular container (via `localToWorld`/`worldToLocal`, set from the `FluidSim` transform in `FluidSim.cs`, line(s) `372--373`) at negligible cost, but it is a purely kinematic, perfectly-reflecting-with-damping wall model: it has no SPH pressure/viscosity interaction with wall particles, so no-slip, free-slip, or partial-slip conditions (study notes lines 500–501) cannot be modeled, and only convex rectangular-box containers are supported — arbitrary solid geometry (e.g. ramps, meshes, moving rigid bodies) is out of scope for this routine.

Two uniform parameters, `edgeForce` and `edgeForceDst` (`FluidSim.compute`, line(s) `44--45`), are declared but *never read* anywhere in `FluidSim.compute`. Likewise, the mouse/pointer interaction uniforms `interactionInputPoint`, `interactionInputStrength`, and `interactionInputRadius` (`FluidSim.compute`, line(s) `51--53`) are declared but never referenced by any kernel. Both appear to be vestigial from a 2D sibling implementation of this simulation (not present in the audited `Simulation` folder) and represent dead/unused code paths here: there is currently no user-interaction force (e.g., a mouse-driven push/pull) and no additional soft edge-repulsion force beyond the hard `ResolveCollisions` clamp.

9 GPU-Specific Structure

Beyond the numerical method itself, the solver is organized as 11 compute kernels dispatched once per sub-step from `RunSimulationStep` (`FluidSim.cs`, line(s) `323--337`): `ExternalForces` → `UpdateSpatialHash` → (GPU sort) → `Reorder` → `ReorderCopyBack` → `CalculateDensities` → `CalculatePressureForce` → (optionally) `CalculateViscosity` → `UpdatePositions`. All buffers are plain `ComputeBuffer`s created and bound once in `Initialize` (`FluidSim.cs`, line(s) `96--255`); there is no dynamic/adaptive resizing of the particle count at runtime (the spawn count is fixed by `Spawner3D.cs` at scene start, `Spawner3D.cs`, line(s) `20--33`). This is an implementation-architecture detail rather than an SPH numerical method, but it bounds what the solver can do (e.g., no particle emitters/sinks, no adaptive refinement) independent of the physics used.

10 Extensions Beyond Classical SPH: Foam, Spray, and Bubbles

A substantial portion of `FluidSim.compute` (`FluidSim.compute`, line(s) `356--607`, inside `CalculatePressureForc` plus the `UpdateWhiteParticles` and `WhiteParticlePrepareNextFrame` kernels, `FluidSim.compute`, line(s) `490--607`) implements a secondary particle system for visual foam/spray/bubble effects, following Ihmsen, Akinci, Gissler, and Teschner's *Unified Spray, Foam and Bubbles for Particle-Based*

Fluids (SIGGRAPH 2012) — a paper not included in the study notes' reading list (Section 5), even though it is a direct companion to the Ihmsen et al. 2014 survey that *is* listed (study notes lines 994–999). Specifically:

- **Trapped-air potential.** For each fluid particle, a "weighted velocity difference" is accumulated from converging neighbor velocity pairs (`FluidSim.compute`, line(s) `313--323`), then combined with a kinetic-energy factor to produce a probabilistic spawn count for new foam/spray particles (`FluidSim.compute`, line(s) `358--366`), sampled over a cylindrical volume swept along the particle's velocity (`FluidSim.compute`, line(s) `368--388`).
- **Classification.** Each foam/spray/bubble particle counts its fluid neighbors in `UpdateWhiteParticles` (`FluidSim.compute`, line(s) `509--540`) and is classified as spray (few neighbors), bubble (many neighbors), or foam (in between) (`FluidSim.compute`, line(s) `542--545`), each with distinct motion rules: foam is advected at the local fluid velocity and dissolves over time (`FluidSim.compute`, line(s) `547--553`); bubbles rise via a buoyancy term and relax toward local fluid velocity (`FluidSim.compute`, line(s) `555--561`); and spray falls under gravity with quadratic drag (`FluidSim.compute`, line(s) `563--568`).
- **Lifetime compaction.** Dead particles are stream-compacted out via an atomic counter and a ping-pong buffer (`WhiteParticlesCompacted`), with the surviving count and buffer copied back for the next frame (`FluidSim.compute`, line(s) `572--607`).

This is a complete, correctly-attributed (if uncredited in-file) 3D implementation of a well-known SPH post-processing technique, but it is visual/decorative: it reads the fluid simulation's state and does not feed back into the pressure/density/viscosity solve (other than the base fluid kernels it reuses for neighbor search).

11 Volumetric Density Resampling

`UpdateDensityTexture` (`FluidSim.compute`, line(s) `478--488`) resamples `CalculateDensitiesAtPoint` on a regular voxel grid sized to the container's aspect ratio (`FluidSim.cs`, line(s) `310--321`, driven by `renderToTex3D`/`densityTextureRes`, `FluidSim.cs`, line(s) `41--42`) and writes the scalar density field into an `RWTexture3D` for external volumetric rendering (e.g. ray marching or isosurfacing, not present in this folder). This reuses the same summation- density kernel already covered in Section 4, evaluated at arbitrary points rather than at particle positions — a direct application of the continuous-reconstruction idea in study notes Section 4.2 (*How Kernel Smoothing Produces a Continuous Approximation*, study notes lines 551–775), $\langle A(\mathbf{x})\rangle \approx \sum_j m_j W(\mathbf{x}-\mathbf{x}_j, h)$ (study notes line 767), evaluated off-particle.

12 Summary Table

SPH Aspect	Implementation	Location
Neighbor search	Uniform grid + spatial hash + GPU counting sort	`SpatialHash3D.hlsl`:1–68; `FluidSim.compute`:146–185; `Helpers/SpatialHash/SpatialHash.cs`:30–36

SPH Aspect	Implementation	Location
Density kernel	Spiky power-2 (non-standard choice, anti-clustering)	`FluidMaths3D.hlsl`:39–47, 69–73
Near-density kernel	Spiky power-3	`FluidMaths3D.hlsl`:28–36, 75–83
Viscosity kernel	Poly6 (used for XSPH, not Laplacian viscosity)	`FluidMaths3D.hlsl`:17–26
Density summation	$\rho_i = \sum_j W_{ij}$, unit mass assumed	`FluidSim.compute`:187–226
Pressure closure	Linearized equation of state $p = (\rho - \rho_0)k$	`FluidSim.compute`:85–88
Near-pressure (stabilization)	Clavet-style second pressure field	`FluidSim.compute`:90–93, 267–346
Pressure force	Symmetrized pairwise force	`FluidSim.compute`:326–332
Viscosity	XSPH velocity smoothing	`FluidSim.compute`:409–459
Time integration	Semi-implicit Euler + predicted positions + fixed sub-steps	`FluidSim.compute`:133–144, 461–475; `FluidSim.cs`:284–308
Boundary handling	Kinematic box clamp + damped reflection	`FluidSim.compute`:101–131
Foam/spray/bubbles	Ihmsen et al. 2012 classification & transport	`FluidSim.compute`:356–388, 490–607
Volumetric field export	Density resampled to 3D texture	`FluidSim.compute`:478–488

13 Not Implemented / Possible Improvements

The following SPH mechanisms are discussed in the study notes (or are standard extensions in the wider literature) but have no corresponding code in `Simulation`:

- **Explicit particle mass.** Study notes lines 758, 811, and 822–827 all carry a per-particle mass m_j through every kernel sum. The implementation has no mass field in any buffer or kernel; it is implicitly 1 everywhere. This only happens to be correct because the spawner (`Spawner3D.cs`) places particles on a uniform grid of constant spacing, so variable-resolution, variable-density, or multi-material setups cannot be represented without first adding a stored mass (or volume) value and threading it through `CalculateDensitiesAtPoint` and `CalculatePressureForce`.

- **Full Tait equation of state / Mach-number-based sound speed.** Study notes lines 262–271 and 433–436 describe choosing c_0 from a target Mach number and using exponent γ (commonly 7 for water). The code's `pressureMultiplier` / `targetDensity` are free-standing stiffness constants with no physical sound-speed derivation, so density-error vs. stiffness trade-offs (study notes lines 433–436) cannot currently be reasoned about quantitatively; exposing an equivalent c_0 and Mach-number readout would make tuning more principled.

- **Incompressible/iterative pressure solvers (ISPH, PCISPH, DFSPH).** Study notes lines 271–292 and the Cummins–Rudman and Solenthaler–Pajarola references (study notes lines 947–949, 973–977) describe enforcing $\nabla \cdot \mathbf{v} = 0$ via a pressure Poisson equation or predictive-corrective iteration. The simulation is purely weakly compressible with a single-pass explicit pressure force, so it is more prone to timestep-limited stiffness and audible/visible compressibility artifacts at

high stiffness; adopting a PCISPH-style predict/correct loop (already partially hinted at by the existing "predicted position" mechanism in Section 7) would be a natural incremental upgrade.

- **Physical viscosity discretization.** Study notes lines 224–232 give the Navier–Stokes viscous term $\mu\nabla^2\mathbf{v}$, and the Morris–Fox–Zhu (1997) reference (study notes lines 940–944) gives a concrete SPH Laplacian for it. Only the non-physical XSPH smoothing (Section 6) is implemented, so `viscosityStrength` does not correspond to a measurable kinematic viscosity ν and cannot be validated against a Reynolds-number-based benchmark (study notes lines 374–378).

- **Artificial viscosity (α, β) and δ-SPH density diffusion.** Study notes lines 452–463 list both as standard stabilizers for shocks and density/pressure noise. Neither is present; the only anti-noise/anti-clustering mechanisms are the near-pressure term (Section 5) and the ad hoc airborne-particle drag (`FluidSim.compute`, line(s) `351--355`).

- **Surface tension.** Study notes Section 3.1 (lines 384–386) and the momentum equation (study notes line 232, $\mathbf{f}_s$) include an interfacial surface-tension force, and it is explicitly one of the features of Müller et al. (2003) cited in the reading list (study notes lines 950–955). No cohesion or curvature force of any kind is computed in `FluidSim.compute`; free-surface behavior is governed solely by the near-pressure anti-clustering term, so droplet break-up/cohesion behavior cannot be tuned independently of overall fluid stiffness.

- **Boundary/ghost particles and no-slip walls.** As discussed in Section 8, only a kinematic box clamp exists; there is no SPH-consistent wall treatment such as Adami–Hu–Adams (2012) (study notes lines 988–992), so wall friction, non-box geometry, and moving/rotating solid obstacles cannot currently be modeled without a substantial boundary-handling addition.

- **CFL-based adaptive timestep.** Study notes lines 469–489 give an explicit formula for Δt from h, c_0, $a_{\max}$, and ν. The code only divides a user-chosen frame time by a fixed `iterationsPerFrame` and clamps the frame time via `maxTimestepFPS` (Section 7); it never measures per-particle velocity or acceleration to adapt the timestep, so a sudden high-energy event (e.g. a large splash) is not automatically guarded against beyond whatever margin the fixed sub-step count happens to provide.

- **Dead/unused interaction and edge-force uniforms.** As noted in Section 8, `edgeForce`, `edgeForceDst`, `interactionInputPoint`, `interactionInputStrength`, and `interactionInputRadius` are declared in `FluidSim.compute`:44–53 but never read. If interactive mouse/controller forcing or a soft edge-repulsion force is desired (both common in SPH demos, including presumably a 2D counterpart to this project), the shader-side uniforms already exist and only need a kernel that consumes them plus CPU-side code to populate them per frame.

- **Adaptive/variable smoothing length.** Study notes lines 408–416 discuss $h = \eta\Delta x$ and adaptive h_i for variable resolution. `smoothingRadius` is a single global scalar (`FluidSim.cs`, line(s) `21`); there is no per-particle h_i or refinement criterion.

- **Energy equation / thermal and multiphase effects.** Study notes Section 2.1.5 (lines 293–313) and the multiphase remarks in Section 2.1.6 are entirely out of scope here, which is reasonable for a single-phase isothermal graphics fluid but worth stating explicitly: there is exactly one fluid phase/material and no temperature field.

- **Convergence/validation tooling.** Study notes Section 3.7 (lines 505–525) recommends refinement studies (halving Δx and Δt, tightening solver tolerances) to confirm results are not

parameter-dependent. No automated convergence test, benchmark scene, or regression harness exists in this folder; adding even a simple dam-break or hydrostatic-pressure test scene with a known analytic or reference result would make future parameter or kernel changes easier to validate.